

优先队列：堆 (Heap)

堆的两条性质

1. 结构性性质：完全二叉树。
2. 堆序性质：最小堆—父 $\leq$ 所有孩子；最大堆—父 $\geq$ 所有孩子。

数组存储（索引从 1 开始）

关系	公式	例子
Parent(i)	$\lfloor i/2 \rfloor$	parent(4)=2
Left(i)	$2i$	left(2)=4
Right(i)	$2i + 1$	right(2)=5

data[0] 作 sentinel（存极小值 MinData），自动终止上滤循环。

考点

这三个公式只用于数组实现的堆！不适用于 BST、链表等其他结构。

注意区分：原地堆排序用索引从 0 开始（left=2i+1, right=2i+2）。

5 种实现的复杂度对比

实现	Insert	DeleteMin
无序数组	$\Theta(1)$	$\Theta(N)$
无序链表	$\Theta(1)$	$\Theta(N)$
有序数组	$O(N)$	$\Theta(1)$
有序链表	$O(N)$	$\Theta(1)$
BST(平衡)	$O(\log N)$	$O(\log N)$
二叉堆	$O(\log N)$	$O(\log N)$

```
h->data[i] = h->data[i/2];    // 父下沉
i = i/2;                      // i上移
}
h->data[i] = x;                // 放好
}
// T(N) = O(log N)
```

手算示例：初始堆 [3,6,5,9,10]，Insert(2)。

2 放 data[6] $\rightarrow$ 跟 data[3]=5 比，5>2，5 下沉到 6，i=3。
跟 data[1]=3 比，3>2，3 下沉到 3，i=1。到根，2 放 data[1]。
结果：[2,6,3,9,10,5]。

初始化

```
PriorityQueue Init(int Max) {
    PriorityQueue H = malloc(sizeof(HeapStruct));
    H->Elements = malloc((Max+1)*sizeof(ElementType));
    H->Capacity = Max; H->Size = 0;
    H->Elements[0] = MinData; // sentinel
    return H;
}
```

易错点

int i = h->size++ 后自增拿的是旧值。必须 ++(h->size)。
循环里写成 data[i/2]=x 做了交换 $\rightarrow$ 应该只下沉，最后才放。
忘了循环外的 h->data[i]=x。

Insert — 上滤 (Percolate Up)

思想：新元素放在末尾（保持完全二叉树），然后和父节点比，小的往上冒。

```
void insert(Heap* h, int x) {
    if(h->size >= MAX-1) return;
    int i = ++(h->size); // 先+后取!
    // 父节点比x大就下沉，空穴上移
    while(i>1 && h->data[i/2] > x) {
```

DeleteMin — 下滤 (Percolate Down)

思想：取走根（最小），最后元素补到根，然后跟较小孩子比，大的往下沉。

```
int deleteMin(Heap* h) {
    if(isEmpty(h)) return -1;
    int min = h->data[1]; // 最小值
    int x = h->data[h->size--]; // 最后一个元素
    int i=1, child;
    while(2*i <= h->size) { // 有左孩子
        child = 2*i;
```

```

// 如果右孩子存在且更小，选右
if(child < h->size && h->data[child+1] < h->data[
    child])
    child++;
if(x <= h->data[child]) break;    // 放好了
h->data[i] = h->data[child];      // 孩子上移
i = child;
}
h->data[i] = x;                  // 放好
return min;
}
// T(N) = O(log N)

```

手算示例：堆 [2,6,3,9,10,5]，DeleteMin。

取走 2，x=5。i=1，child=2(左) vs 3(右)，3 更小。5>3，3 上移到 1，i=3。

i=3，child=6(左)，2*i=6 > size=5，无孩子，停。5 放在 3。结果：[3,6,5,9,10]。

其他堆操作

- **DecreaseKey(P, Δ , H)**：减小某键值 $\rightarrow$ 上滤。提高进程优先级。
- **IncreaseKey(P, Δ , H)**：增大某键值 $\rightarrow$ 下滤。降低 CPU 占用。
- **Delete(P, H)**：DecreaseKey(P, ∞ , H) 使其到根 + DeleteMin(H)。
- **BuildHeap(H)**：Floyd 自底向上建堆 $O(N)$ 。

BuildHeap —Floyd 法 $O(N)$

思想：从最后一个非叶节点 ($N/2$) 开始，往前逐个下滤。叶子不用处理。

```

void percdDown(Heap* h, int i) {
    int x = h->data[i], child;
    while(2*i <= h->size) {
        child = 2*i;
        if(child < h->size && h->data[child+1] < h->data[
            child])
            child++;
        if(x <= h->data[child]) break;
        h->data[i] = h->data[child];
        i = child;
    }
    h->data[i] = x;
}

void buildHeap(Heap* h, int arr[], int N) {
    for(int i=0; i<N; i++) h->data[i+1] = arr[i];
    h->size = N;
    for(int j = N/2; j >= 1; j--)          // 从最后一个非叶
        to根
        percdDown(h, j);
}
// T(N) = O(N) — 不是 O(N log N)!

```

考点

为什么 BuildHeap 是 $O(N)$ 不是 $O(N \log N)$?

定理：完美二叉树中所有节点高度和 = $2^{h+1}-1-(h+1) \approx N$ 。

大部分节点在底层 (叶子) 几乎不下沉，总工作量接近 N 而非 $N \log N$ 。

手算示例：arr=[4,2,7,1,3,6,5]，N=7。N/2=3。

下滤 data[3]=7：7 和 5 换 $\rightarrow$ [4,2,5,1,3,6,7]。

下滤 data[2]=2：跟孩子 1 和 3 比，1 < 2 $\rightarrow$ 2 和 1 换 $\rightarrow$ [4,1,5,2,3,6,7]。

下滤 data[1]=4：跟 1 和 5 比，1 < 4 $\rightarrow$ 4 和 1 换 $\rightarrow$ [1,4,5,2,3,6,7]。再跟 2 和 3 比，2 < 4 $\rightarrow$ 4 和 2 换 $\rightarrow$ [1,2,5,4,3,6,7]。

易错点

for(j=N/2; j>1; j--) 漏了根 j=1。应为 j>=1。

拷数组写了 arr+i (指针地址) $\rightarrow$ 应 arr[i]。

h->size 用 for 循环 ++ $\rightarrow$ 直接 h->size=N。

Floyd 写成 while(j!=1)+j/=2+percdDown 两下 $\rightarrow$ 应是简单 for。

易错点

大小比较反了：最小堆找较小孩子，最大堆找较大孩子。
child!=h->size 条件漏了 $\rightarrow$ 可能访问越界。
忘了最后 h->data[i]=x。

d-堆

- DeleteMin 比较次数 $d - 1$ 增多

每个节点有 d 个孩子。DeleteMin 需 $d-1$ 次比较选最小 $\rightarrow O(d \log_d N)$ 。

索引关系： $\text{parent} = \lfloor (i + d - 2) / d \rfloor$,
 $\text{children} = d(i - 1) + 2, d(i - 1) + 3, \dots, di + 1$ 。

- 大 d 的优缺点：
- + 树更矮，外存场景（磁盘 I/O 少）
 - $*d / /d$ 不是位运算（对比 $*2 / /2$ 是移位）

应用：找第 k 大元素

方法	复杂度
排序取第 k	$O(N \log N)$
大小 k 的最小堆	$O(N \log k)$
BuildHeap + k 次 DeleteMax	$O(N + k \log N)$
QuickSelect	平均 $O(N)$ ，最坏 $O(N^2)$